

Guía de Laboratorio 2

Entornos Virtuales en Python: Creación y Buenas Prácticas

Módulo introductorio para Data Science Programming

1 Introducción: ¿Por qué necesitamos entornos virtuales?

Cuando trabajas en varios proyectos de ciencia de datos al mismo tiempo, es muy probable que cada uno dependa de versiones distintas —y a veces incompatibles— de las mismas librerías. Por ejemplo, un proyecto de clase puede requerir `pandas 1.5`, mientras que un proyecto de investigación más reciente necesita `pandas 2.2`. Si instalas todo directamente sobre el Python global de tu computador, tarde o temprano un proyecto terminará rompiendo al otro.

Un **entorno virtual** (*virtual environment*, o `venv`) resuelve este problema: es una copia aislada del intérprete de Python, con su propia carpeta de librerías, completamente independiente del Python del sistema operativo y de los demás proyectos que tengas en tu computador. Piensa en cada entorno virtual como una caja de herramientas exclusiva para un proyecto: lo que instalas o modificas dentro de una caja jamás afecta a las demás.

- **Sin entornos virtuales:** un solo Python global y un solo conjunto de librerías compartido por todos tus proyectos → conflictos de versiones, el clásico “ayer funcionaba”, y proyectos difíciles de reproducir en otro computador.
- **Con entornos virtuales:** cada proyecto tiene su propio Python y sus propias librerías → cero conflictos entre proyectos, instalaciones reproducibles, y la libertad de borrar y recrear un entorno sin miedo a dañar el resto del sistema.

¡Importante!

¿**POR QUÉ ES OBLIGATORIO Y NO OPCIONAL?** En versiones recientes de Python (3.11+), los sistemas basados en Debian/Ubuntu y macOS (con Homebrew) protegen el Python del sistema operativo. Si intentas instalar una librería con `pip install` sin un entorno virtual activo, es muy probable que recibas el siguiente error:

```
error: externally-managed-environment
```

Este mensaje no es un error tuyo: es una medida de seguridad para evitar que rompas el Python que el propio sistema operativo necesita para funcionar internamente. La solución **no** es forzar la instalación con `--break-system-packages`, sino trabajar siempre dentro de un entorno virtual, tal como se explica en esta guía.

2 Creación de un Entorno Virtual

Desde Python 3.3, el módulo `venv` viene incluido de forma nativa en el intérprete, por lo que no necesitas instalar ninguna herramienta adicional (más allá de lo configurado en la **Guía de Laboratorio 1**).

2.1 Windows

1. Abre una terminal (*Símbolo del sistema* o *PowerShell*) dentro de la carpeta de tu proyecto.
2. Ejecuta el siguiente comando, donde `venv` es el nombre que le darás a la carpeta del entorno:

```
1 python -m venv venv
```

2.2 macOS / Linux

1. Abre una terminal dentro de la carpeta de tu proyecto.
2. Ejecuta el mismo comando, usando `python3` en lugar de `python`:

```
1 python3 -m venv venv
```

¡Importante!

CONVENCIÓN DE NOMBRES: Puedes llamar a la carpeta del entorno como quieras, pero `venv` y `.venv` son las convenciones más usadas en la industria. En macOS y Linux, el punto inicial (`.venv`) oculta la carpeta en los listados normales del explorador de archivos, manteniendo tu proyecto más limpio visualmente. Evita usar espacios o tildes en el nombre del entorno.

Al finalizar, se creará una nueva carpeta dentro de tu proyecto que contiene una copia del intérprete de Python y las herramientas base (`pip`, `setuptools`). Todavía **no está activo**: para usarlo, debes activarlo explícitamente, como se explica a continuación.

3 Activación y Desactivación del Entorno

Activar un entorno virtual le indica a tu terminal (y a VS Code) que, mientras esté activo, todos los comandos `python` y `pip` deben usar el intérprete y las librerías de esa carpeta, en lugar de las del sistema operativo.

Acción	Windows	macOS / Linux
Crear el entorno	<code>python -m venv venv</code>	<code>python3 -m venv venv</code>
Activar (CMD)	<code>venv\Scripts\activate.bat</code>	—
Activar (PowerShell)	<code>venv\Scripts\Activate.ps1</code>	—
Activar (bash / zsh)	—	<code>source venv/bin/activate</code>
Desactivar	<code>deactivate</code>	

3.1 Windows

1. Si usas el *Símbolo del sistema* (CMD), ejecuta:

```
1 venv\Scripts\activate.bat
```

2. Si usas *PowerShell*, ejecuta en su lugar:

```
1 venv\Scripts\Activate.ps1
```

¡Importante!

ERROR COMÚN EN POWERSHELL: Si al ejecutar `Activate.ps1` PowerShell responde con un mensaje similar a “*no se puede cargar el archivo ... porque la ejecución de scripts está deshabilitada en este sistema*”, es porque Windows restringe por defecto la ejecución de scripts de PowerShell por seguridad. Para habilitarla (una sola vez por usuario), abre PowerShell **como administrador** y ejecuta:

```
1 Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

Confirma escribiendo **S** (o **Y**) y presiona Enter. Si prefieres no modificar esta configuración, simplemente activa el entorno usando el *Símbolo del sistema* (CMD), donde este problema no ocurre.

3.2 macOS / Linux

1. Desde la terminal, dentro de la carpeta de tu proyecto, ejecuta:

```
1 source venv/bin/activate
```

3.3 Verificación y Desactivación

- **¿Cómo sé que está activo?** El nombre del entorno aparecerá entre paréntesis al inicio de la línea de tu terminal, por ejemplo: `(venv) C:\proyecto>` o `(venv) usuario@equipo:~/proyecto$`.
- **Confirmación adicional:** puedes ejecutar `where python` (Windows) o `which python3` (macOS/Linux); la ruta mostrada debe apuntar **dentro** de la carpeta `venv` de tu proyecto, no a una ruta del sistema.
- **Para desactivar** el entorno en cualquier sistema operativo, simplemente ejecuta:

```
1 deactivate
```

4 Integración con Visual Studio Code

VS Code está diseñado para trabajar de la mano con entornos virtuales, pero es importante indicarle explícitamente cuál usar.

1. Al abrir una carpeta de proyecto que contiene una carpeta `venv` (o `.venv`), VS Code normalmente muestra una notificación en la esquina inferior derecha preguntando si deseas usarla como intérprete. Acepta la sugerencia.
2. Si la notificación no aparece, selecciónalo manualmente: presiona `Ctrl+Shift+P` (`Cmd+Shift+P` en macOS) para abrir la *Paleta de Comandos*, escribe **“Python: Select Interpreter”** y elige la opción que muestre la ruta (`'venv': venv`).
3. A partir de ese momento, cada vez que abras una nueva terminal integrada en VS Code (*Terminal > New Terminal*), el entorno se activará automáticamente.

¡Importante!

JUPYTER NOTEBOOKS: Seleccionar el intérprete de Python **no** selecciona automáticamente el kernel de un *notebook* (`.ipynb`). Dentro del notebook, haz clic en el selector de kernel (esquina superior derecha) y elige explícitamente el que corresponda a tu `venv`. Si instalas una librería nueva y el notebook no la reconoce, casi siempre es porque el kernel seleccionado no es el de tu entorno virtual.

5 La Importancia de Gestionar Bien tus Entornos

Crear un entorno virtual es solo la mitad del trabajo: gestionarlo correctamente durante todo el proyecto es lo que realmente evita dolores de cabeza más adelante.

- **Un entorno por proyecto:** nunca reutilices el mismo `venv` para varios proyectos distintos; pierdes exactamente el aislamiento que buscabas al crearlo.
- **Nunca subas la carpeta `venv` a Git:** puede pesar cientos de megabytes, contiene rutas específicas de tu computador y es completamente innecesaria para tus compañeros de equipo. En su lugar, agrega esta línea a tu archivo `.gitignore`:

```
1 venv/
```

- **Comparte dependencias, no el entorno completo.** Con el entorno activo, genera un archivo de requisitos:

```
1 pip freeze > requirements.txt
```

Cualquier persona (o tú mismo, en otro computador) puede recrear exactamente las mismas librerías con:

```
1 pip install -r requirements.txt
```

- **Ante la duda, recrea en lugar de reparar:** si un entorno queda en un estado confuso (librerías a medio instalar, versiones mezcladas), casi siempre es más rápido borrar la carpeta `venv` y crearla de nuevo que intentar depurarla manualmente.
- **Actualiza tu `requirements.txt`** cada vez que instales o quites una librería importante del proyecto, no solo al final.

6 Prueba de Fuego: Ciclo Completo de un Entorno Virtual

Sigue estos pasos dentro de la carpeta `Laboratorio.Datos` que creaste en la Guía de Laboratorio 1, para practicar el ciclo completo de vida de un `venv`.

1. **Crea el entorno** (usa el comando correspondiente a tu sistema operativo, según la Sección 2).
2. **Actívalo** (según la Sección 3) y confirma que tu terminal muestra el prefijo (`venv`).
3. **Verifica que `pip` apunta al entorno correcto:**

```
1 pip --version
```

La ruta que aparece junto a la versión debe estar dentro de tu carpeta `venv`.

4. **Instala Pandas dentro del entorno:**

```
1 pip install pandas
```



5. Congela las dependencias en un archivo:

```
1 pip freeze > requirements.txt
```

Abre el archivo generado y observa que contiene, como mínimo, **pandas** junto con sus dependencias (por ejemplo **numpy**).

6. Desactiva el entorno:

```
1 deactivate
```

7. **Comprueba la reproducibilidad.** Borra por completo la carpeta **venv**, créala de nuevo (paso 1), actívala (paso 2) y, en lugar de instalar Pandas manualmente, ejecuta:

```
1 pip install -r requirements.txt
```

Si Pandas queda instalado correctamente sin que tuvieras que recordar el nombre exacto de la librería, ¡acabas de comprobar por qué gestionar bien tu entorno virtual hace que tu trabajo sea reproducible!